IT24103114

MUTHUMALI R.M.P.V.

IA LAB 1

Part 1: Code Analysis & PEAS Evaluation

Task 1.1: The Environment State (__init__)

Q1: List the four components of the PEAS framework discussed in Lecture 01.

Answer:

- Performance Measure
- Environment
- Actuators
- Sensors

Q2: Look at the VisualGridHuntGame.__init__ method. Which specific variables in this code represent the physical "Environment (E)" state?

Answer:

- Grid dimensions: self.width, self.height

- Walls / Obstacles: self.walls
- Food positions: self.food_positions
- Opponent positions: self.opponents
- Current Agent position: self.agent_pos

Q3: Based on self.opponents, classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Answer:

Multi-Agent Environment.

- Justification: The code initializes self.opponents which move around dynamically in the grid and can collide with the main agent, directly impacting its score and state.

Task 1.2: The Perception Subsystem (get_percept)

Q4: Define what a "percept sequence" is according to Lecture 01.

Answer: A percept sequence is the complete history of everything the agent has perceived/sensed from its environment up to the current state.

Q5: Which component of the PEAS framework does the get_percept() method represent in our code?

Answer: Sensors (S) (It gathers and packages the environment state into a percept dictionary for the agent).

Q6: Based on the exact dictionary returned by get_percept(), is this environment "Fully Observable" or "Partially Observable"? Explain why.

Answer:

Partially Observable.

- Explanation: The agent does not receive the complete set of wall coordinates (self.walls) in its percept dictionary. It can only sense whether it has already hit a wall ('hit_wall') or if food/opponents are at its current position, meaning it lacks full global map visibility.

Task 1.3: Action Execution (execute_action)

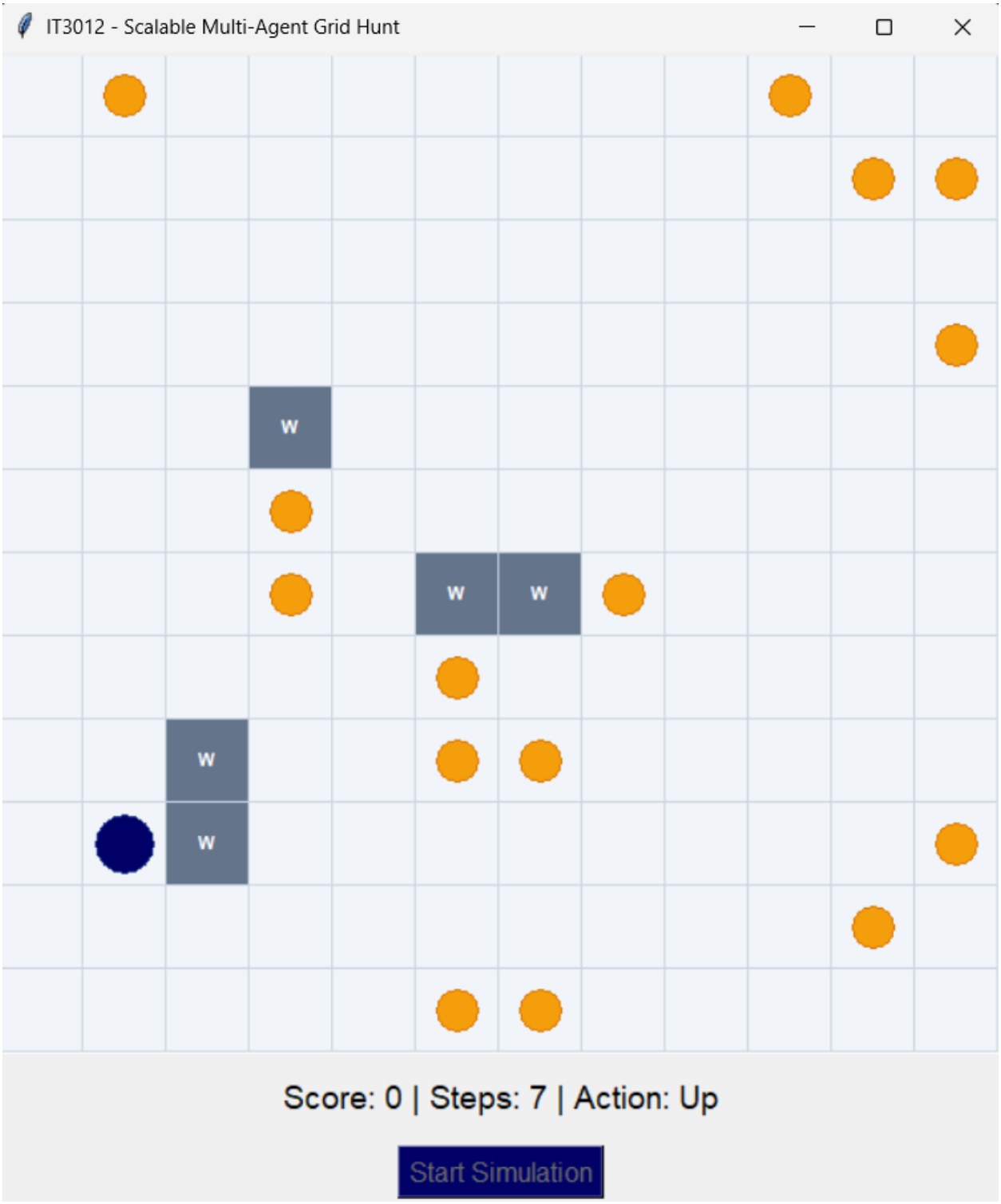
Q7: When execute_action() deducts points for hitting a wall, which PEAS component is being actively updated?

Answer: Performance Measure (P) (It modifies self.score based on the agent's actions and collisions)

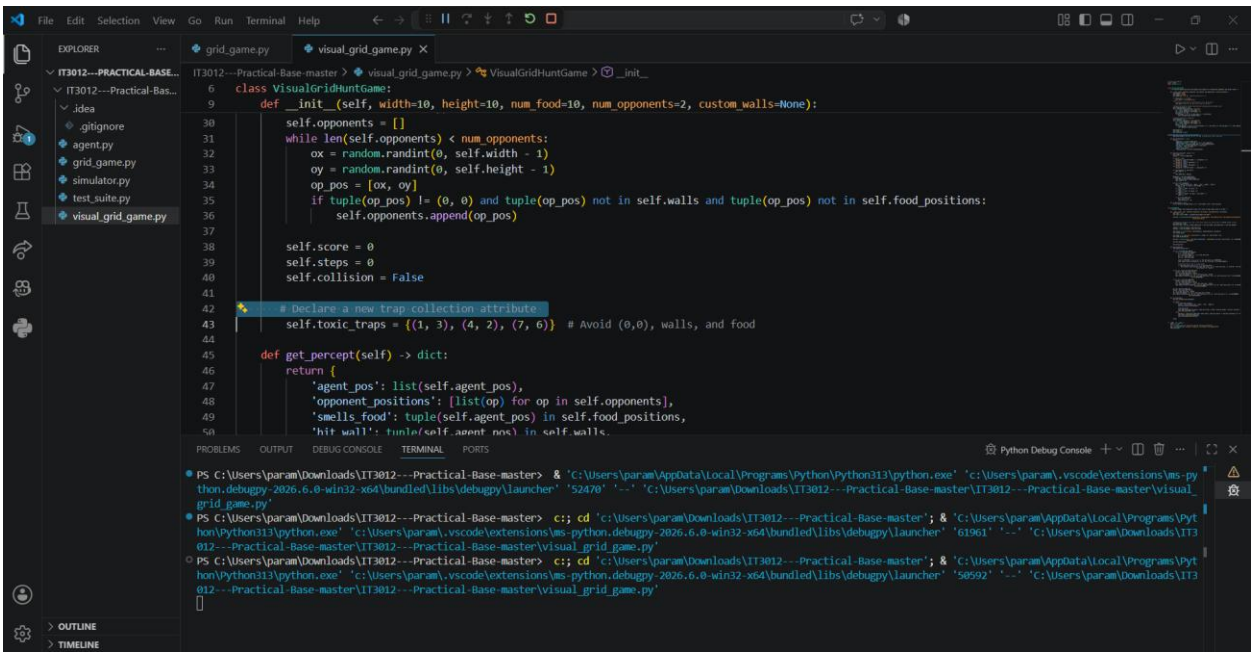
Q8: Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Answer: An agent's rationality and success must be evaluated based on the actual outcome/impact produced in the environment (e.g., collecting food or avoiding walls), not on how much computing power or logic the agent used internally. Measuring external state prevents "metric gaming" and guarantees task utility optimization.

Part 2: Guided Modification & Deep Theory Mapping



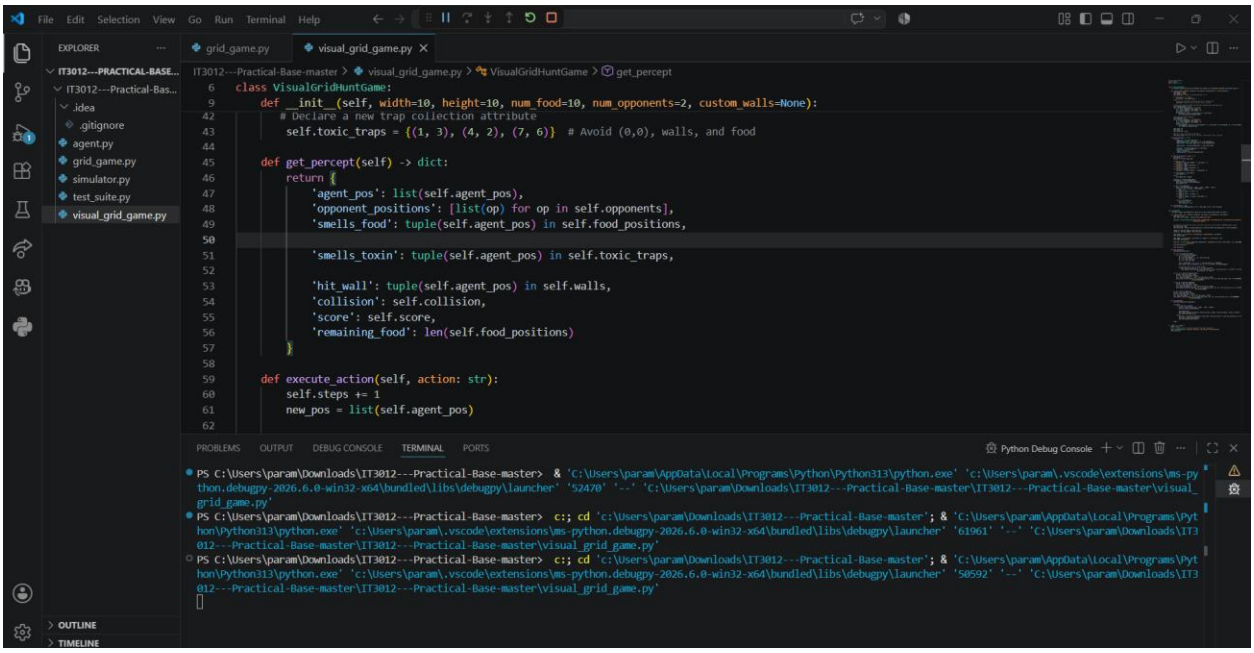
Step 2.1: change the __init__ (Declare a new trap collection attribute)



Q9: If you successfully add self.toxic_traps to the environment but intentionally hide this data from the agent's sensors, how does this alter the environment's "Observability"?

Answer: The environment becomes Partially Observable (or increases in partial unobservability), because a crucial environment component (toxic_traps) exists in the state space but cannot be perceived by the agent prior to stepping on it.

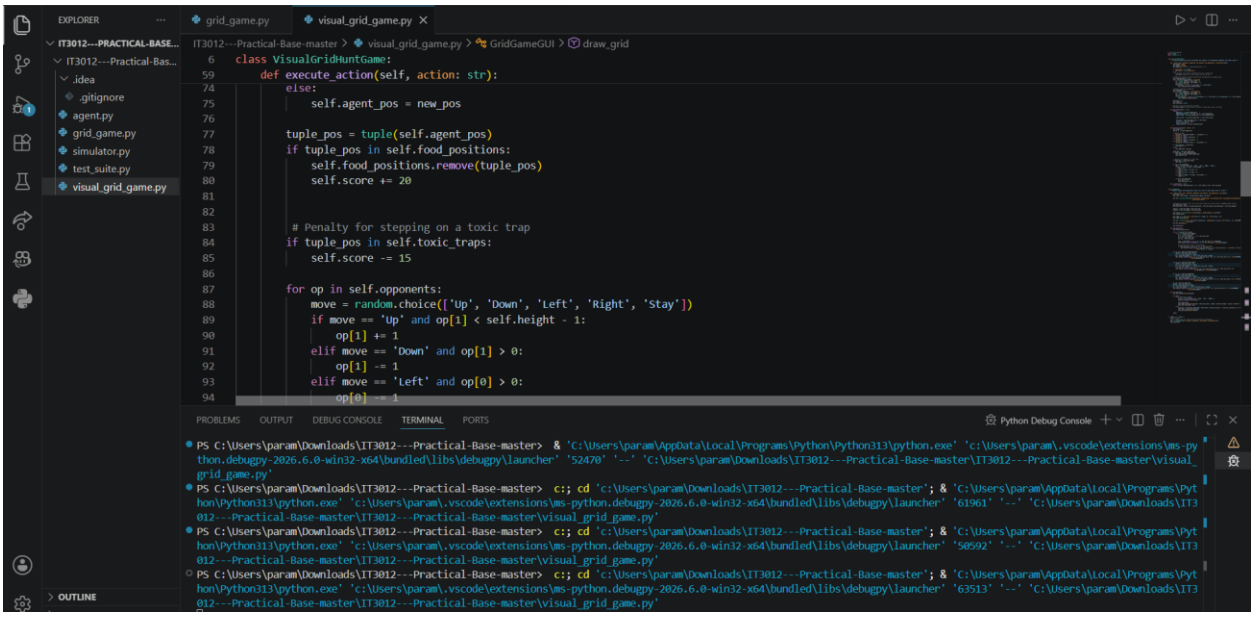
Step 2.2: Updating the Perception Subsystem (get_percept)

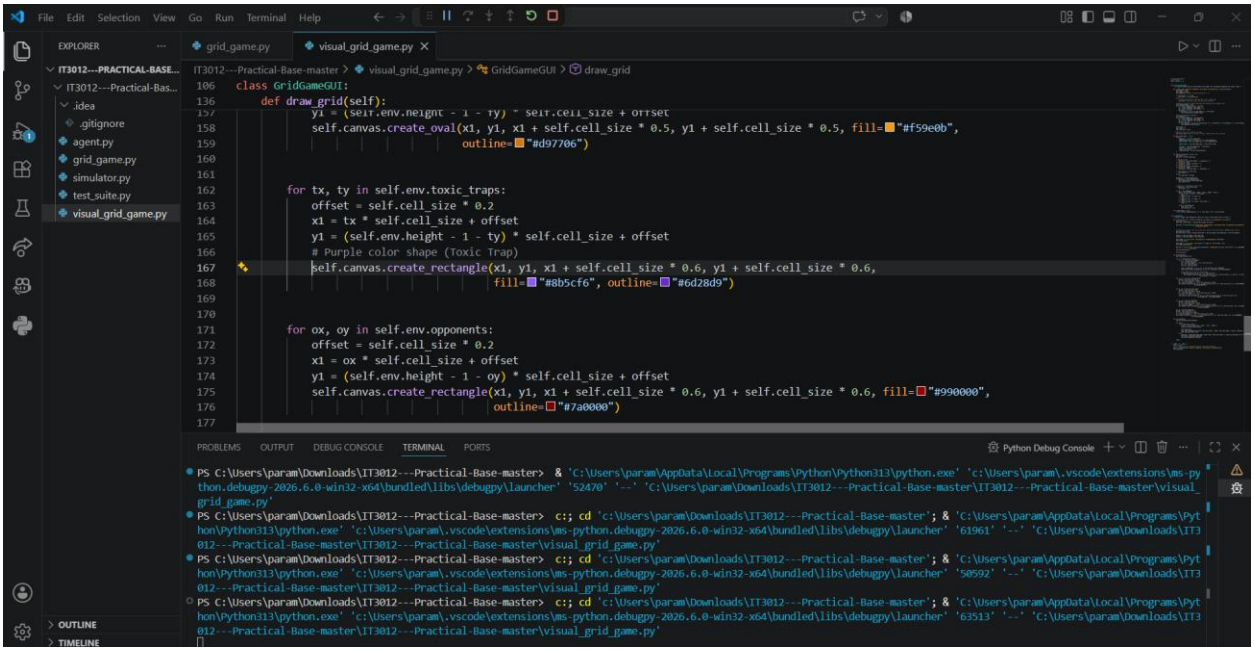


Q10: Explain how adding 'smells_toxin' sensor helps the agent act with "Rationality" (maximizing expected utility).

Answer: Rationality requires an agent to select actions that maximize its expected utility based on its percept history. By receiving the 'smells_toxin' sensor input, the agent gets awareness of hazard positions, allowing it to choose safer actions that avoid the severe -15 point penalty and thus optimize its overall score.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)





Q11: You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

Answer: In the Vacuum Cleaner World, if the performance measure rewards the agent simply for "cleaning dirt", a poorly designed agent can dump collected dirt back onto the floor and clean it again repeatedly. This exploits the metric to gain infinite points without fulfilling the true objective (keeping the environment clean).